

FILE: app_entrypoint.py

```
import tkinter as tk
import sys
import os
from pillow_heif import register_heif_opener
# Регистрируем поддержку HEIC для Pillow
register_heif_opener()
# Корректная работа путей внутри EXE
if getattr(sys, 'frozen', False):
    basedir = sys._MEIPASS
else:
    basedir = os.path.dirname(__file__)
sys.path.append(basedir)
# Импорты наших модулей
from app_ui import SetupWindow, PhotoSorterApp
from core_engine import PhotoEngine
from data_config import load_settings, ensure_dirs
def start_main(config):
    """Запуск основного приложения после настройки"""
    # Очищаем все временные виджеты (окна настройки и т.д.)
    for w in root.winfo_children():
        w.destroy()
    # Показываем и разворачиваем окно
    root.deiconify()
    try:
        root.state('zoomed')
    except:
        root.geometry("1200x800")
        root.update()
    # Запускаем само приложение
    PhotoSorterApp(root, config, engine)
    if __name__ == "__main__":
        # Импортируем новую функцию cleanup_cache_smart
        from data_config import ensure_dirs, cleanup_cache_smart
        # 1. Создаем необходимые папки (.photo_cache и .photo_trash)
        ensure_dirs()
        # 2. Умная очистка: удалит файлы старше 7 дней.
        # Если кэш всё равно больше 0.5 ГБ — удалит самые старые до этого лимита.
        cleanup_cache_smart(days_limit=7, max_gb=0.5)
        root = tk.Tk()
        root.title("Photo Sorter Pro")
        root.configure(bg="#121212")
        engine = PhotoEngine()
        config = load_settings()
        # Проверяем настройки. Если источника нет — сначала окно Setup.
        if not config or not config.get("source") or not os.path.exists(config.get("source")):
            root.withdraw()
            SetupWindow(root, start_main)
        else:
            start_main(config)
        root.mainloop()
```

FILE: app_ui.py

```
import threading
import tkinter as tk
from tkinter import filedialog, messagebox
from PIL import Image, ImageTk, ImageOps
import os
import shutil
from data_config import save_settings, load_settings, THUMB_SIZE, TRASH_DIR
from app_filmstrip import PhotoFilmstrip
class SetupWindow(tk.Toplevel):
    def __init__(self, parent, on_success):
        super().__init__(parent)
        self.on_success = on_success
        self.title("Настройка путей")
```

```

self.geometry("650x480")
self.grab_set()
saved = load_settings() or {}
self.paths = {k: tk.StringVar(value=saved.get(k, ""))
for k in ["source"] + [{"dest{i}"} for i in range(1, 7)]}
for k, txt in [{"source", "ИСТОЧНИК:"}] + [{"dest{i}"} for i in range(1, 7)]:
f = tk.Frame(self); f.pack(fill=tk.X, padx=15, pady=6)
tk.Label(f, text=txt, width=15, anchor="w", font=("Arial", 9, "bold")).pack(side=tk.LEFT)
tk.Entry(f, textvariable=self.paths[k], bg="#f9f9f9").pack(side=tk.LEFT, expand=True, fill=tk.X, padx=5)
tk.Button(f, text="Обзор", command=lambda x=k: self.browse(x)).pack(side=tk.RIGHT)
tk.Button(self, text="СОХРАНИТЬ", bg="#2e7d32", fg="white", font=("Arial", 11, "bold"), command=self.finish).pack(pady=20)
def browse(self, key):
self.grab_release()
path = filedialog.askdirectory(parent=self)
if path: self.paths[key].set(path)
self.grab_set()
def finish(self):
res = {k: v.get() for k, v in self.paths.items()}
if res["source"]:
save_settings(res)
if self.on_success: self.on_success(res)
self.destroy()
class PhotoSorterApp:
def __init__(self, root, config, engine):
self.root, self.config, self.engine = root, config, engine
self.current_idx = 0
self.selected_indices = {}
self.files = []
self.current_photo_tk = None
self.history = []
self.last_nav_time = 0 # Для умной задержки
self.engine.current_source = self.config.get('source', "")
self._init_ui()
self._bind_keys()
self.root.protocol("WM_DELETE_WINDOW", self.on_app_closing)
threading.Thread(target=self._async_load_files, daemon=True).start()
self._check_queue()
def _init_ui(self):
self.root.columnconfigure(0, weight=1)
self.root.rowconfigure(1, weight=1)
# ВЕРХНЯЯ ИНФО-ПАНЕЛЬ
self.info_panel = tk.Frame(self.root, bg="#1a1a1a", height=30)
self.info_panel.grid(row=0, column=0, sticky="ew")
self.info_panel.pack_propagate(False)
self.fname_label = tk.Label(self.info_panel, text="", fg="#00e5ff", bg="#1a1a1a",
font=("Consolas", 10, "bold"))
self.fname_label.pack(side=tk.LEFT, padx=10)
self.cache_stat = tk.Label(self.info_panel, text="□ READY", fg="#00e5ff",
bg="#1a1a1a", font=("Consolas", 9, "bold"), width=15)
self.cache_stat.pack(side=tk.RIGHT, padx=10)
# Главная область просмотра
self.main_label = tk.Label(self.root, bg="#121212", text="") # Добавили text=""
self.main_label.grid(row=1, column=0, sticky="nsew")
# Полоска прогресса
self.prog_frame = tk.Frame(self.root, height=2, bg="#222222")
self.prog_frame.grid(row=2, column=0, sticky="ew")
self.prog_bar = tk.Frame(self.prog_frame, bg="#00e5ff", width=0, height=2)
self.prog_bar.place(x=0, y=0)
self.filmstrip = PhotoFilmstrip(self.root, self.engine, self.on_thumb_click)
# Панель управления
self.ctrl = tk.Frame(self.root, bg="eeeeee", pady=5)
self.ctrl.grid(row=4, column=0, sticky="ew")
self.btn_l = tk.Frame(self.ctrl, bg="eeeeee")
self.btn_l.pack(side=tk.LEFT, padx=10)
tk.Button(self.btn_l, text="□", command=self.open_settings).pack(side=tk.LEFT, padx=2)
tk.Button(self.btn_l, text="□ Undo", command=self.undo_last_action).pack(side=tk.LEFT, padx=2)
tk.Button(self.btn_l, text="□", command=lambda: self.navigate(-1)).pack(side=tk.LEFT, padx=2)
tk.Button(self.btn_l, text="□", command=lambda: self.navigate(1)).pack(side=tk.LEFT, padx=2)
tk.Button(self.btn_l, text="□ Поворот (R)", command=self.rotate_current).pack(side=tk.LEFT, padx=10)
tk.Button(self.btn_l, text="□ Удалить", fg="red", command=self.delete_files).pack(side=tk.LEFT, padx=2)

```

```

self.btn_r = tk.Frame(self.ctrl, bg="#eeeeee")
self.btn_r.pack(side=tk.RIGHT, padx=10)
self.btns_container = tk.Frame(self.btn_r, bg="#eeeeee")
self.btns_container.pack(side=tk.LEFT)
self.refresh_buttons()
def _bind_keys(self):
self.root.bind("<Left>", lambda e: self.navigate(-1))
self.root.bind("<Right>", lambda e: self.navigate(1))
self.root.bind("<Delete>", lambda e: self.delete_files())
self.root.bind("<Control-z>", lambda e: self.undo_last_action())
self.root.bind("<Control-Z>", lambda e: self.undo_last_action())
self.root.bind("r", lambda e: self.rotate_current())
self.root.bind("R", lambda e: self.rotate_current())
self.root.bind("к", lambda e: self.rotate_current())
self.root.bind("К", lambda e: self.rotate_current())
for i in range(1, 7):
self.root.bind(str(i), lambda e, x=i: self.move_action(x))
self.root.bind("<Configure>", lambda e: self.root.after(100, self.refresh_ui) if e.widget == self.root else None)
def _async_load_files(self):
try:
source = self.config.get('source', "")
files = self.engine.get_file_list(source)
self.root.after(0, self._on_files_loaded, files)
except:
self.root.after(0, self._on_files_loaded, [])
def _on_files_loaded(self, files):
"""Вызывается в главном потоке, когда список файлов готов"""
self.files = files
if not self.files:
# Если папка пуста
self.main_label.config(text="Папка пуста или не содержит фото", image="")
self.fname_label.config(text="")
self.filmstrip._clear_all()
else:
# 1. Устанавливаем начальные индексы
self.current_idx = 0
self.selected_indices = {0}
# 2. ЗАПУСКАЕМ ФОНОВОЕ КЭШИРОВАНИЕ ВСЕЙ ПАПКИ
# Мы запускаем это в отдельном потоке, чтобы само создание очереди задач
# не подтормаживало интерфейс при очень больших списках (1000+ фото)
threading.Thread(
target=self.engine.precache_all,
args=(self.files,),
daemon=True
).start()
# 3. Обновляем интерфейс (показываем первое фото и ленту)
self.refresh_ui()
# Принудительно проталкиваем обновление заголовка и текста
display_text = f"[{self.current_idx + 1} / {len(self.files)}] {self.files[self.current_idx]}"
self.fname_label.config(text=display_text)
self.root.title(f"Photo Sorter Pro 2.1 | {display_text}")
# Убираем возможные "зависшие" надписи
self.root.update_idletasks()
def undo_last_action(self):
if not self.history: return
action = self.history.pop()
restored = []
for old_path, new_path, fname in action['items']:
try:
if os.path.exists(new_path):
shutil.move(new_path, old_path)
restored.append(fname)
except: pass
if restored:
self.files.extend(restored)
self.files.sort()
self.current_idx = self.files.index(restored[0])
self.selected_indices = {self.current_idx}
self._after_file_list_change()
def _save_previous_rotations(self):

```

```

fnames = list(self.engine.rotation_map.keys())
if not fnames: return
current_fname = self.files[self.current_idx] if self.files else ""
for f in fnames:
    if f != current_fname:
        self.engine.executor.submit(self.engine.save_rotation_to_disk, f)
def navigate(self, step):
    if not self.files: return
    self._save_previous_rotations()
    self.current_idx = (self.current_idx + step) % len(self.files)
    self.selected_indices = {self.current_idx}
    # 1. Мгновенно обновляем текст и заголовок
    display_text = f"[{self.current_idx + 1} / {len(self.files)}] — {self.files[self.current_idx]}"
    self.fname_label.config(text=display_text)
    self.root.title(f"Photo Sorter Pro 2.1 | {display_text}")
    # 2. Обновляем ленту
    self.filmstrip.refresh(self.files, self.current_idx, self.selected_indices, self.root.winfo_width())
    # 3. АНТИ-ФРИЗ: Большое фото грузим ТОЛЬКО после паузы в 150мс
    if hasattr(self, '_nav_after_id'):
        self.root.after_cancel(self._nav_after_id)
    # Если "летим", main_label не трогаем, чтобы не вешать поток
    self._nav_after_id = self.root.after(150, self.show_current)
    def _check_queue(self):
        # 1. Получаем нагрузку
        q_size = self.engine.get_queue_size()
        # 2. Мгновенно обновляем текст счётчика
        if q_size > 0:
            self.cache_stat.config(text=f"▣ LOAD: {q_size}", fg="#ff9800")
        else:
            self.cache_stat.config(text="▣ READY", fg="#00e5ff")
        # ПРИНУДИТЕЛЬНОЕ ОБНОВЛЕНИЕ ЭКРАНА (чтобы счётчик не замерзл)
        self.cache_stat.update_idletasks()
        # 3. Обработка готовых миниатюр
        updated = False
        while not self.engine.result_queue.empty():
            try:
                self.engine.result_queue.get_nowait()
                updated = True
            except: break
        if updated:
            self.filmstrip.refresh(self.files, self.current_idx, self.selected_indices, self.root.winfo_width())
    # 4. Прогресс-бар
    if self.files and self.root.winfo_width() > 10:
        progress = (self.current_idx + 1) / len(self.files)
        try: self.prog_bar.config(width=int(self.root.winfo_width() * progress))
        except: pass
    # Ставим интервал 50мс для максимальной отзывчивости
    self.root.after(50, self._check_queue)
    def on_thumb_click(self, idx, state):
        self._save_previous_rotations()
        if state & 0x0001:
            start, end = min(self.current_idx, idx), max(self.current_idx, idx)
            for i in range(start, end + 1): self.selected_indices.add(i)
        elif state & 0x0004:
            if idx in self.selected_indices: self.selected_indices.remove(idx)
            else: self.selected_indices.add(idx)
        else:
            self.selected_indices = {idx}
            self.current_idx = idx
            self.refresh_ui()
    def rotate_current(self):
        if not self.files: return
        for idx in self.selected_indices:
            if idx < len(self.files):
                self.engine.rotate_in_memory(self.files[idx])
            self.refresh_ui()
    def show_current(self):
        if not self.files or self.current_idx >= len(self.files): return
        fname = self.files[self.current_idx]
        path = os.path.join(self.config['source'], fname)

```

```

# Инфо-текст (номер и имя)
display_text = f"[{self.current_idx + 1} / {len(self.files)}] — {fname}"
# Выводим инфо ТОЛЬКО в черную панель над фото
self.fname_label.config(text=display_text)
# Заголовок окна теперь всегда чистый и статичный
self.root.title("Photo Sorter Pro 2.1")
# Очищаем текст в центре (чтобы не было дублей)
self.main_label.config(text="")
def load_full():
try:
with Image.open(path) as img:
img = ImageOps.exif_transpose(img)
angle = self.engine.rotation_map.get(fname, 0)
if angle != 0:
img = img.rotate(angle, expand=True)
w, h = max(100, self.main_label.winfo_width()), max(100, self.main_label.winfo_height())
img.thumbnail((w, h), Image.Resampling.LANCZOS)
self.current_photo_tk = ImageTk.PhotoImage(img)
# Устанавливаем фото, текст принудительно пустой
self.main_label.config(image=self.current_photo_tk, text="")
except:
self.main_label.config(text="Ошибка загрузки", image="")
threading.Thread(target=load_full, daemon=True).start()
def move_action(self, folder_num):
dest_path = self.config.get(f"dest{folder_num}")
if not dest_path or not self.files: return
to_move = [self.files[i] for i in sorted(self.selected_indices, reverse=True) if i < len(self.files)]
history_item = {'type': 'move', 'items': []}
for fname in to_move:
src = os.path.join(self.config['source'], fname)
dst = os.path.join(dest_path, fname)
try:
self.engine.save_rotation_to_disk(fname)
shutil.move(src, dst)
history_item['items'].append((src, dst, fname))
self.files.remove(fname)
except: pass
if history_item['items']: self.history.append(history_item)
self._after_file_list_change()
def delete_files(self):
if not self.selected_indices or not self.files: return
if not messagebox.askyesno("Удаление", "Удалить выбранные?"): return
to_delete = [self.files[i] for i in sorted(self.selected_indices, reverse=True) if i < len(self.files)]
trash_path = os.path.abspath(TRASH_DIR)
if not os.path.exists(trash_path): os.makedirs(trash_path)
history_item = {'type': 'delete', 'items': []}
for fname in to_delete:
src = os.path.join(self.config['source'], fname)
dst = os.path.join(trash_path, fname)
try:
self.engine.clear_cache([fname])
shutil.move(src, dst)
history_item['items'].append((src, dst, fname))
self.files.remove(fname)
except: pass
if history_item['items']: self.history.append(history_item)
self._after_file_list_change()
def _after_file_list_change(self):
self.filmstrip._clear_all()
self.selected_indices = set()
if not self.files:
self.current_idx = 0
self.main_label.config(image="", text="Панка пуста")
self.fname_label.config(text="")
else:
if self.current_idx >= len(self.files): self.current_idx = len(self.files) - 1
self.selected_indices = {self.current_idx}
self.refresh_ui()
def open_settings(self):
SetupWindow(self.root, self.on_settings_changed)

```

```
def on_settings_changed(self, new_config):
self.config = new_config
self.engine.current_source = self.config.get('source', '')
self.current_idx, self.files, self.history = 0, [], []
self.engine.clear_cache()
self.filmstrip._clear_all()
self.root.update_idletasks()
self.refresh_buttons()
threading.Thread(target=self._async_load_files, daemon=True).start()
def refresh_buttons(self):
for widget in self.btns_container.winfo_children(): widget.destroy()
if any(self.config.get(f"dest{i}") for i in range(1, 7)):
tk.Label(self.btns_container, text="B nanky:", bg="#eeeeee", font=("Arial", 9, "bold")).pack(side=tk.LEFT, padx=5)
for i in range(1, 7):
path = self.config.get(f"dest{i}", "")
if path:
name = os.path.basename(path.rstrip(os.sep))
btn = tk.Button(self.btns_container, text=f"{i}: {name}", padx=8,
command=lambda x=i: self.move_action(x), font=("Arial", 9, "bold"),
bg="#ffffff", relief=tk.GROOVE)
btn.pack(side=tk.LEFT, padx=2)
def refresh_ui(self):
if not self.files: return
self.show_current()
self.filmstrip.refresh(self.files, self.current_idx, self.selected_indices, self.root.winfo_width())
def on_app_closing(self):
"""Экстренное и полное завершение всех процессов"""
try:
# 1. Останавливаем движок
self.engine.stop_engine()
# 2. Сохраняем последние повороты (быстро)
frames = list(self.engine.rotation_map.keys())
for f in frames:
self.engine.save_rotation_to_disk(f)
except:
pass
finally:
# 3. Убиваем окно
self.root.destroy()
# 4. САМОЕ ВАЖНОЕ: Полный выход из Python
# Это мгновенно приберет все зависшие фоновые потоки
os._exit(0)
```

```
import os
import threading
import queue
import hashlib

from concurrent.futures import ThreadPoolExecutor
from PIL import Image, ImageTk, ImageOps
from data_config import THUMB_SIZE, EXTENSIONS, MAX_WORKERS, CACHE_DIR, ensure_dirs

class PhotoEngine:
    def __init__(self):
        self.thumb_cache = {}
        self.current_source = ""
        self.lock = threading.Lock()
        self.result_queue = queue.Queue()
        self.executor = ThreadPoolExecutor(max_workers=MAX_WORKERS)
        self.rotation_map = {} # Храним углы поворота {имя_файла: угол}
        ensure_dirs() # Создаем .photo_cache и .photo_trash
        def get_file_list(self, source_dir):
            if not source_dir or not os.path.exists(source_dir):
                return []
            self.current_source = os.path.abspath(source_dir)
            try:
                return sorted([f for f in os.listdir(self.current_source)
                               if f.lower().endswith(EXTENSIONS)])
            except:
```

```

except:
    return []
def _get_cache_path(self, fname):
    hash_name = hashlib.md5(fname.encode()).hexdigest()
    return os.path.join(CACHE_DIR, f"{hash_name}.png")
def get_thumb(self, fname):
    with self.lock:
        return self.thumb_cache.get(fname)
def request_thumb(self, fname):
    """Загрузка превью с защитой от перегрузки очереди"""
    current_load = self.get_queue_size()
    # Если в очереди уже больше 60 задач, новые пока не принимаем
    if current_load < 60:
        self.executor.submit(self._proc_thumb, fname)
def precache_all(self, files):
    """Закидывает всю папку в очередь на фоновую загрузку миниатюр"""
    for fname in files:
        # Проверяем, нет ли уже этого файла в RAM-кэше
        with self.lock:
            if fname in self.thumb_cache:
                continue
        # Постепенно наполняем пул потоков
        self.executor.submit(self._proc_thumb, fname)
def stop_engine(self):
    """Останавливает пул потоков немедленно"""
    # shutdown(wait=False) говорит потокам "бросайте всё"
    self.executor.shutdown(wait=False, cancel_futures=True)
def _proc_thumb(self, fname):
    with self.lock:
        if fname in self.thumb_cache:
            self.result_queue.put(fname)
            return
        cache_path = self._get_cache_path(fname)
        path = os.path.join(self.current_source, fname)
        img_to_show = None
        try:
            # Пытаемся открыть из кэша
            if os.path.exists(cache_path):
                try:
                    with Image.open(cache_path) as cached_img:
                        img_to_show = cached_img.copy() # Копируем в память и СРАЗУ закрываем файл
                        img_to_show.load()
                except:
                    img_to_show = None
                try: os.remove(cache_path)
                except: pass # Если файл занят, просто пропустим удаление в этот раз
            # Если кэша нет - создаем
            if img_to_show is None:
                with Image.open(path) as original:
                    original.draft('RGB', (THUMB_SIZE, THUMB_SIZE))
                    img_to_show = ImageOps.exif_transpose(original)
                    img_to_show.thumbnail((THUMB_SIZE, THUMB_SIZE), Image.Resampling.LANCZOS)
                    img_to_show.save(cache_path, "PNG")
            # Поворот в памяти
            angle = self.rotation_map.get(fname, 0)
            if angle != 0:
                img_to_show = img_to_show.rotate(angle, expand=True)
            photo = ImageTk.PhotoImage(img_to_show)
            with self.lock:
                self.thumb_cache[fname] = photo
                self.result_queue.put(fname)
        except Exception as e:
            print(f"Ошибка миниатюры {fname}: {e}")
def rotate_in_memory(self, fname):
    """Мгновенный поворот в памяти"""
    current_angle = self.rotation_map.get(fname, 0)
    new_angle = (current_angle - 90) % 360
    self.rotation_map[fname] = new_angle
    with self.lock:
        self.thumb_cache.pop(fname, None)

```

```

return new_angle
def save_rotation_to_disk(self, fname):
    """Физическая запись на диск. Вызывается в фоне."""
    angle = self.rotation_map.pop(fname, 0)
    if angle == 0: return
    path = os.path.join(self.current_source, fname)
    try:
        # Небольшая пауза, чтобы файл освободился GUI-поток
        import time
        time.sleep(0.1)
        with Image.open(path) as img:
            img = ImageOps.exif_transpose(img)
            rotated = img.rotate(angle, expand=True)
            rotated.save(path, quality=95, subsampling=0)
        cache_path = self._get_cache_path(fname)
        if os.path.exists(cache_path):
            try: os.remove(cache_path)
            except: pass
        except Exception as e:
            print(f"Ошибка сохранения {fname}: {e}")
    def clear_cache(self, filenames=None):
        with self.lock:
            if filenames:
                for f in filenames: self.thumb_cache.pop(f, None)
            else:
                self.thumb_cache.clear()
    def get_queue_size(self):
        """Возвращает количество задач в очереди пула потоков"""
        try:
            # Обращаемся к внутренней очереди исполнителя
            return self.executor._work_queue.qsize()
        except:
            return 0

```

FILE: app_filmstrip.py

```

import tkinter as tk
from data_config import THUMB_SIZE
class PhotoFilmstrip:
    def __init__(self, parent, engine, on_click_callback):
        self.engine = engine
        self.on_click_callback = on_click_callback
        self.start_idx = 0
        self.t_widgets = []
        # Храним последнее состояние, чтобы не перерисовывать лишнее
        self.last_state = {}
        # Контейнер ленты
        self.frame = tk.Frame(parent, height=THUMB_SIZE + 20, bg="#1e1e1e")
        self.frame.grid(row=2, column=0, sticky="ew")
        self.frame.pack_propagate(False)
        # Создаем 45 постоянных слотов (пул виджетов)
        for _ in range(45):
            c = tk.Frame(self.frame, width=THUMB_SIZE + 6, height=THUMB_SIZE + 6, bg="#1e1e1e")
            c.pack(side=tk.LEFT, padx=2, pady=5)
            c.pack_propagate(False)
            l = tk.Label(c, bg="#1e1e1e")
            l.place(relx=0.5, rely=0.5, anchor="center")
            self.t_widgets.append((c, l))
        def refresh(self, files, current_idx, selected_indices, win_width):
            """Оптимизированная отрисовка ленты"""
            if not files or current_idx >= len(files): # Добавлена проверка индекса
                self._clear_all()
            return
            if current_idx >= len(files):
                return
            # 1. Расчет видимого диапазона (сколько влезет в ширину окна)
            num_vis = min(len(self.t_widgets), max(1, win_width // (THUMB_SIZE + 10)))
            # Автоматическая прокрутка ленты за курсором

```



```

if current_idx >= self.start_idx + num_vis:
self.start_idx = current_idx - num_vis + 1
elif current_idx < self.start_idx:
self.start_idx = current_idx
if self.start_idx + num_vis > len(files):
self.start_idx = max(0, len(files) - num_vis)
# 2. Обновление только изменившихся виджетов
for i, (frame, label) in enumerate(self.t_widgets):
idx = self.start_idx + i
if idx < len(files):
fname = files[idx]
is_active = (idx == current_idx or idx in selected_indices)
bg_color = "#ffd600" if is_active else "#1e1e1e"
# Обновляем цвет только если он изменился
if frame.cget("bg") != bg_color:
frame.config(bg=bg_color)
label.config(bg=bg_color)
thumb = self.engine.get_thumb(fname)
# Обновляем картинку только если она появилась в кэше
# и не совпадает с тем, что уже установлено в Label
if thumb:
if label.cget("image") != str(thumb):
label.config(image=thumb)
else:
label.config(image="")
self.engine.request_thumb(fname)
# Перепривязываем события (необходимо для актуального индекса)
cb = lambda e, x=idx: self.on_click_callback(x, e.state)
label.bind("<Button-1>", cb)
frame.bind("<Button-1>", cb)
else:
self._clear_slot(frame, label)
def _clear_slot(self, frame, label):
if label.cget("image") or frame.cget("bg") != "#1e1e1e":
label.config(image="", bg="#1e1e1e")
frame.config(bg="#1e1e1e")
label.unbind("<Button-1>")
frame.unbind("<Button-1>")
def _clear_all(self):
for c, l in self.t_widgets:
self._clear_slot(c, l)

```

FILE: data_config.py

```

import json
import os
import multiprocessing
import time
SETTINGS_FILE = "photo_sorter_settings.json"
CACHE_DIR = ".photo_cache"
TRASH_DIR = ".photo_trash" # Папка для временного хранения удаленных фото
THUMB_SIZE = 160
EXTENSIONS = ('.jpg', '.jpeg', '.png', '.bmp', '.webp', '.heic', '.heif')
# Оптимальное кол-во потоков
MAX_WORKERS = multiprocessing.cpu_count() + 2
def save_settings(settings):
with open(SETTINGS_FILE, "w", encoding="utf-8") as f:
json.dump(settings, f, ensure_ascii=False, indent=4)
def load_settings():
if os.path.exists(SETTINGS_FILE):
try:
with open(SETTINGS_FILE, "r", encoding="utf-8") as f:
return json.load(f)
except:
return None
return None
def ensure_dirs():
"""Создает все системные папки при запуске"""

```

```

for d in [CACHE_DIR, TRASH_DIR]:
    if not os.path.exists(d):
        try:
            os.makedirs(d)
        except:
            pass
    def get_dir_size(path):
        """Считает размер папки в байтах"""
        total = 0
        try:
            with os.scandir(path) as it:
                for entry in it:
                    if entry.is_file():
                        total += entry.stat().size
        except:
            pass
        return total
    def cleanup_cache_smart(days_limit=7, max_gb=0.5):
        """
        Умная очистка кэша:
        1. Удаляет файлы, к которым не обращались более 7 дней.
        2. Если после этого кэш все еще больше 500МБ, удаляет самые старые до лимита.
        """
        if not os.path.exists(CACHE_DIR):
            return
        now = time.time()
        seconds_limit = days_limit * 24 * 60 * 60
        limit_bytes = max_gb * 1024 * 1024 * 1024
        files_data = []
        current_size = 0
        # Собираем данные о файлах
        try:
            for f in os.listdir(CACHE_DIR):
                p = os.path.join(CACHE_DIR, f)
                if os.path.isfile(p):
                    stat = os.stat(p)
                    # Используем st_atime (время последнего доступа)
                    files_data.append({
                        'path': p,
                        'atime': stat.st_atime,
                        'size': stat.st_size
                    })
                current_size += stat.st_size
        except:
            return
        # 1. Удаляем файлы старше лимита по дням
        deleted_count = 0
        remaining_files = []
        for item in files_data:
            if (now - item['atime']) > seconds_limit:
                try:
                    os.remove(item['path'])
                    current_size -= item['size']
                    deleted_count += 1
                except:
                    pass
            else:
                remaining_files.append(item)
        # 2. Если кэш все еще слишком большой, чистим по объему (самые старые сначала)
        if current_size > limit_bytes:
            remaining_files.sort(key=lambda x: x['atime'])
            target_size = limit_bytes * 0.7
            for item in remaining_files:
                try:
                    os.remove(item['path'])
                    current_size -= item['size']
                    deleted_count += 1
                except:
                    pass
            if current_size <= target_size:
                break
        if deleted_count > 0:
            print(f"Очистка кэша: удалено {deleted_count} файлов. Текущий размер: {current_size // 1024 // 1024} МБ")

```

FILE: requirements.txt

```
# Основная библиотека для обработки изображений
Pillow>=10.0.0
# Поддержка форматов Apple HEIC/HEIF
pillow-heif>=0.13.0
# Инструмент для сборки приложения в EXE
pyinstaller>=6.0.0
```

FILE: readme.md

```
# □ Photo Sorter Pro v2.3.1 (Modular)
![Python](https://shields.io)
![Pillow](https://shields.io)
![Format](https://shields.io)
**Photo Sorter Pro** — это профессиональный инструмент для высокоскоростной сортировки, просмотра и базовой обработки фотоархивов. Разработан для эффективной работы с тысячами снимков.
---
## □ Ключевые возможности версии 2.3.1
* □ **HEIC/HEIF Support**: Полная поддержка современных форматов Apple (iPhone) благодаря интеграции `pillow-heif`.
* □ **Full Background Precache**: При открытии папки приложение автоматически запускает фоновую генерацию миниатюр для всей директории.
* □ **Real-time Load Monitoring**: Счетчик в реальном времени (`LOAD: N`) показывает количество задач в очереди обработки.
* □ **Smart Cache Manager**:
**Time-based logic**: Удаление кэша, к которому не обращались более 7 дней.
**Safe Limit**: Ограничитель на 500 МБ для предотвращения избыточного использования дискового пространства.
* □ □ **Undo System**: Возможность отмены перемещений и удалений. Файлы восстанавливаются из локальной корзины `.photo_trash`.
* □ **Clean Exit**: Принудительное завершение всех фоновых потоков при закрытии программы, исключающее "зависание" процесса в диспетчере задач.
---
## □ □ Управление (Hotkeys)
| Действие | Клавиша | Описание |
| :--- | :--- | :--- |
| **Навигация** | `←` / `→` | Листать фотографии (адаптивная задержка при зажатии) |
| **Сортировка** | `1` - `6` | Переместить в соответствующую папку |
| **Поворот** | `R` | Физический поворот файла (поддержка любой раскладки) |
| **Удаление** | `Delete` | Безопасное удаление в локальную корзину |
| **Отмена** | `Ctrl + Z` | Undo: вернуть файл на место (**только EN раскладка**) |
---
## □ Архитектура проекта
- `app_entrypoint.py`: Точка входа. Инициализация HEIC-декодера и проверка лимитов кэша.
- `app_ui.py`: Интерфейс, верхняя инфо-панель, логика навигации и система Undo.
- `core_engine.py`: Многопоточный движок на базе `ThreadPoolExecutor`.
- `app_filmstrip.py`: Оптимизированная лента миниатюр.
- `data_config.py`: Конфигурация, системные пути и алгоритмы очистки.
---
## □ Установка и сборка
### Установка зависимостей
```bash
pip install -r requirements.txt
Сборка в EXE
pyinstaller --noconfirm --onefile --windowed --icon="icon.ico" --name "PhotoSorterPro_2.3" --clean app_entrypoint.py
```